

>

💡 WHY ADVANCED DATA STRUCTURES?

Standard data structures break down under dynamic constraints. Advanced structures combine multiple techniques (e.g., Doubly Linked List + HashMap for LRU Cache) to guarantee ****O(1) or O(log N) operations**** under complex workloads!

💡 THE INVERSE ACKERMANN AHA

Disjoint Set Union (DSU) with ****Path Compression**** and ****Union by Rank**** reduces `find()` and `union()` operations to near-constant $O(\alpha(N)) \approx O(1)$ time!

1 4 CORE ADVANCED STRUCTURES

- **Union-Find (DSU):** Dynamic connectivity, cycle detection in $O(\alpha(N))$.
- **Segment Tree:** Range minimum / sum queries & point updates in $O(\log N)$.
- **Fenwick Tree (BIT):** Prefix sums and updates in $O(\log N)$ with zero tree overhead.
- **LRU Cache:** Doubly Linked List + HashMap for $O(1)$ get/put eviction.

2 VISUALIZING PATH COMPRESSION

Before vs After Path Compression:

Before: 4 → 3 → 2 → 1 (root)
find(4) flattens tree!
After: 4 → 1, 3 → 1, 2 → 1
Tree height becomes 1!

📚 PREREQUISITES & COMPLEXITY REASONING

Data Structure	Query Time	Update Time	Space
Union-Find (DSU)	$O(\alpha(N)) \approx O(1)$	$O(\alpha(N)) \approx O(1)$	$O(N)$
Segment Tree	$O(\log N)$ range	$O(\log N)$ point	$O(4N)$ array
Fenwick Tree (BIT)	$O(\log N)$ prefix	$O(\log N)$ point	$O(N)$ array
LRU Cache	$O(1)$ get	$O(1)$ put	$O(\text{Capacity})$

🔥 CLUES THAT SCREAM ADVANCED DS

1. "Dynamic graph connectivity / Detect cycle in undirected graph" → Union-Find (DSU)

2. "Dynamic range sum queries with point updates" → Segment Tree or Fenwick Tree

3. "Design O(1) cache with least recently used eviction" → LRU Cache (DLL + Map)

>

1 OPTIMAL UNION-FIND (DSU) TEMPLATE

```
class DSU {
    int[] parent, rank;
    public DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    public int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path Compression!
    }
    public boolean union(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI == rootJ) return false; // Cycle detected!
        if (rank[rootI] < rank[rootJ]) parent[rootI] = rootJ;
        else if (rank[rootI] > rank[rootJ]) parent[rootJ] = rootI;
        else { parent[rootJ] = rootI; rank[rootI]++; }
        return true;
    }
}
```

2 FENWICK TREE (BINARY INDEXED TREE)

```
class FenwickTree {
    int[] tree;
    public FenwickTree(int n) { tree = new int[n + 1]; }
    public void add(int i, int delta) {
        for (i++; i < tree.length; i += i & -i) tree[i] += delta;
    }
    public int query(int i) {
        int sum = 0;
        for (i++; i > 0; i -= i & -i) sum += tree[i];
        return sum;
    }
    public int rangeQuery(int l, int r) {
        return query(r) - query(l - 1);
    }
}
```

>

PATTERN1

UNION-FIND CYCLE DETECTION (Redundant Connection)

e.g. Redundant Connection (LC 684), Number of Connected Components

CYCLE DETECTION MECHANICS

For each edge (u, v) , if `union(u, v) == false` (meaning `find(u) == find(v)`), the current edge creates a cycle and is redundant!

TEMPLATE — LC 684

```
public int[] findRedundantConnection(int[][] edges) {
    int n = edges.length;
    DSU dsu = new DSU(n + 1);
    for (int[] edge : edges) {
        if (!dsu.union(edge[0], edge[1])) return edge; // Cycle edge found!
    }
    return new int[0];
}
```

>

PATTERN2

SEGMENT TREE (Range Sum Query & Point Update)

e.g. Range Sum Query - Mutable (LC 307)

SEGMENT TREE ARRAY SIZE $4N$

Divide range in half recursively. Left child $= 2i+1$, Right child $= 2i+2$. Point updates & range sum queries run in $O(\log N)$!

TEMPLATE — LC 307

```
class NumArray {
    int[] tree, nums; int n;
    public NumArray(int[] nums) {
        this.nums = nums; n = nums.length;
        if (n > 0) { tree = new int[4 * n]; build(0, 0, n - 1); }
    }
    private void build(int node, int start, int end) {
        if (start == end) { tree[node] = nums[start]; return; }
        int mid = (start + end) / 2;
        build(2 * node + 1, start, mid); build(2 * node + 2, mid + 1, end);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }
    public void update(int index, int val) { updateTree(0, 0, n - 1, index, val); }
    private void updateTree(int node, int start, int end, int idx, int val) {
        if (start == end) { nums[idx] = val; tree[node] = val; return; }
        int mid = (start + end) / 2;
        if (start <= idx && idx <= mid) updateTree(2 * node + 1, start, mid, idx, val);
        else updateTree(2 * node + 2, mid + 1, end, idx, val);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }
    public int sumRange(int left, int right) { return query(0, 0, n - 1, left, right); }
    private int query(int node, int start, int end, int L, int R) {
        if (R < start || end < L) return 0;
        if (L <= start && end <= R) return tree[node];
        int mid = (start + end) / 2;
        return query(2 * node + 1, start, mid, L, R) + query(2 * node + 2, mid + 1, end, L, R);
    }
}
```

>

PATTERN 3

LRU CACHE (Doubly Linked List + HashMap)

e.g. LRU Cache (LC 146)

SENTINEL HEAD & TAIL GUARD
HashMap maps `key → Node`. Dummy `head` and `tail` nodes eliminate edge case checks! Most recently used node is moved to head; least recently used at tail is evicted!

TEMPLATE — LC 146

```
class LRUCache {
    class Node { int key, val; Node prev, next; Node(int k, int v) { key=k; val=v; } }
    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0, 0), tail = new Node(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity; head.next = tail; tail.prev = head;
    }
    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key); remove(node); insertToHead(node);
        return node.val;
    }
    public void put(int key, int value) {
        if (map.containsKey(key)) remove(map.get(key));
        if (map.size() == capacity) { map.remove(tail.prev.key); remove(tail.prev); }
        Node node = new Node(key, value); insertToHead(node); map.put(key, node);
    }
    private void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void insertToHead(Node n) { n.next = head.next; n.next.prev = n; head.next = n; n.prev = head; }
}
```

>

PATTERN 4

LFU CACHE (Frequency HashMap + Frequency Lists)

e.g. LFU Cache (LC 460)

MIN FREQUENCY STATE
Map `key → Node`. Map `freq → DoublyLinkedList`. Maintain `minFreq`. When capacity reached, evict from `freqMap.get(minFreq).tail.prev`!

TEMPLATE — LC 460

```
class LFUCache {
    class Node { int key, val, freq = 1; Node prev, next; Node(int k, int v) { key=k; val=v; } }
    class DoubleLinkedList {
        Node head = new Node(0, 0), tail = new Node(0, 0); int size = 0;
        DoubleLinkedList() { head.next = tail; tail.prev = head; }
        void add(Node n) { n.next = head.next; n.next.prev = n; head.next = n; n.prev = head; size++; }
        void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev; size--; }
        Node removeLast() { if (size == 0) return null; Node n = tail.prev; remove(n); return n; }
    }
    int cap, minFreq = 0;
    Map<Integer, Node> keyMap = new HashMap<>();
    Map<Integer, DoubleLinkedList> freqMap = new HashMap<>();

    public LFUCache(int capacity) { cap = capacity; }
    public int get(int key) {
        if (!keyMap.containsKey(key)) return -1;
        Node node = keyMap.get(key); updateFreq(node); return node.val;
    }
    private void updateFreq(Node n) {
        DoubleLinkedList oldList = freqMap.get(n.freq); oldList.remove(n);
        if (n.freq == minFreq && oldList.size == 0) minFreq++;
        n.freq++;
        freqMap.computeIfAbsent(n.freq, k → new DoubleLinkedList()).add(n);
    }
}
```

>

ADVANCED DS — WHICH STRUCTURE TO USE?

Start: What is the problem requirement?

Dynamic graph components & cycle checks

→

Union-Find (DSU)
Path compression & rank

Range sum/min queries with point updates

→

Segment Tree / Fenwick Tree
 $O(\log N)$ updates & range queries

$O(1)$ Cache eviction by least recent

→

LRU Cache
HashMap + Doubly Linked List

$O(1)$ Cache eviction by least frequent

→

LFU Cache
Frequency Map + DLL Lists

TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Redundant edge / Cycle detection"	Union-Find DSU	If <code>find(u) == find(v)</code> , cycle!
"Mutable range sum"	Segment / BIT	Tree update & query in $O(\log N)$
"LRU Cache $O(1)$ "	DLL + HashMap	Move to head on <code>get</code> , evict tail
"LFU Cache $O(1)$ "	Freq Map + DLL	Maintain <code>minFreq</code> and update list

INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Segment Tree vs Fenwick Tree?	Fenwick Tree (BIT) uses 3x less memory and is easier to code for RANGE SUMS. Segment Tree handles generic operations (min/max/GCD)!
Why Doubly Linked List for LRU Cache?	A Doubly Linked List allows $O(1)$ deletion given a node pointer (<code>node.prev.next = node.next</code>), which a singly linked list cannot do!

MEDIUM — DSU & Range Sums

LC 684 · Redundant Connection

MEDIUM

Pattern: DSU Cycle Check

Key Insight: Return edge if `union == false`.

```
public int[] findRedundantConnection(int[][] edges) {
    DSU dsu = new DSU(edges.length + 1);
    for (int[] e : edges)
        if (!dsu.union(e[0], e[1])) return e;
    return new int[0];
}
```

LC 307 · Range Sum Query - Mutable

MEDIUM

Pattern: Fenwick Tree (BIT)

Key Insight: `add()` and `query()` with `i & -i`.

```
class NumArray {
    int[] tree, nums;
    public NumArray(int[] nums) {
        this.nums = nums; tree = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) add(i, nums[i]);
    }
    public void update(int i, int val) { add(i, val - nums[i]); nums[i] = val; }
    private void add(int i, int d) {
        for (; i < tree.length; i += i & -i) tree[i] += d;
    }
    public int sumRange(int l, int r) { return q(r) - q(l - 1); }
    private int q(int i) { int s = 0; for (; i > 0; i -= i & -i) s += tree[i]; return s; }
}
```

HARD — Cache Design

LC 146 · LRU Cache

MEDIUM

Pattern: Map + DLL

Key Insight: Move to head on `get`, evict tail on full.

```
public int get(int key) {
    if (!map.containsKey(key)) return -1;
    Node n = map.get(key); remove(n); insertHead(n); return n.val;
}
```

LC 460 · LFU Cache

HARD

Pattern: Freq Map + DLL

Key Insight: Evict from `freqMap.get(minFreq)`.

```
private void updateFreq(Node n) {
    DoubleLinkedList oldL = freqMap.get(n.freq); oldL.remove(n);
    if (n.freq == minFreq && oldL.size == 0) minFreq++;
    n.freq++; freqMap.computeIfAbsent(n.freq, k -> new DoubleLinkedList()).add(n);
}
```

COMPLETE ADVANCED DATA STRUCTURES PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
684	Redundant Connection	Union-Find DSU	$O(N \cdot \alpha(N))$	$O(N)$	M
307	Range Sum Query - Mutable	Fenwick Tree / Segment Tree	$O(\log N)$ query/update	$O(N)$	M
146	LRU Cache	Doubly Linked List + Map	$O(1)$ get/put	$O(\text{Capacity})$	M
460	LFU Cache	Frequency Map + DLL Lists	$O(1)$ get/put	$O(\text{Capacity})$	H

MASTERING ADVANCED STRUCTURES — QUICK SUMMARY

- DSU:** Use Path Compression (`parent[i] = find(parent[i])`) and Union by Rank for $O(\alpha(N))$ time.
- Fenwick Tree (BIT):** Use lowbit `i & -i` to advance or retreat index. `add()` moves forward, `query()` moves backward.
- LRU & LFU Caches:** Always use dummy `head` and `tail` nodes in your Doubly Linked List to eliminate `null` pointer checks during node removal!

>

 DRY RUN — DSU Path Compression (`find(4)`)

Step	Node	parent[i] Before	Action / Compression	parent[i] After
1	4	3	Recurse <code>find(3)</code>	1
2	3	2	Recurse <code>find(2)</code>	1
3	2	1	Recurse <code>find(1)</code> ↳ Root is 1!	1
4	4	3	<code>parent[4] = 1</code> (Direct link to root!)	1 

 MATHEMATICAL PROOF — LOWBIT OPERATOR (`i & -i`)

Claim: `i & -i` isolates the lowest set bit (power of 2) of integer `i`.

Proof: `-i` is computed as two's complement `~i + 1`. Inverting `i` flips all bits. Adding 1 flips trailing 1s back to 0 until reaching the first 0 (which was the lowest 1 in `i`). Performing `i & (~i + 1)` cancels all higher bits ↳ **Exact Lowest Bit Power of 2!**

 MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write DSU with Path Compression & Union by Rank

☐ Code Redundant Connection cycle check

☐ Implement Fenwick Tree (BIT) add & query

☐ Code Segment Tree build, update, & range query

☐ Implement LRU Cache (DLL + HashMap) in 10m

☐ Write LFU Cache min-frequency map eviction

☐ Use dummy head/tail sentinels for DLL

☐ Prove why `i & -i` extracts lowbit

 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Perform Path Compression

In DSU `find()`, write `return parent[i] = find(parent[i])`! Without assignment, trees can degrade to $O(N)$ height!

Rule 2 — Always Check Map Size in LRU Put

In `LRUCache.put()`, if key ALREADY EXISTS, remove old node first BEFORE checking if `map.size() == capacity`!